

COMS W4156 Advanced Software Engineering (ASE)

October 1, 2024

On this day ...



Pelé's Final Game

1977 Brazilian soccer great Pelé plays his final game for the New York Cosmos in an exhibition against Santos in front of 75,000 at Giants Stadium; 1,281 goals in 1,363 games



Football Legend

[Pele](#)

Mentor Assignments Posted

See <https://static.us.edusercontent.com/files/cyH1Otdpf1wAZy6KfGjdmz0c>



Agenda

1. Distributed Computing communication patterns
2. RPC vs gRPC
3. gRPC demo: Justin Tan
4. Next



www.shutterstock.com · 1993681

Computing

One computer



Caller makes a regular function call to the callee, providing parameters if required by the function signature, e.g., `int foo (int x, int y)` requires passing two integers

The callee (`foo`) receives the parameters, performs some computation, and returns result if required by the function signature, e.g., `int foo (int x, int y)` requires returning an `int`

The caller and the callee run on the same computer and do not need to know about any other computers



(Simple) Distributed Computing

Two or more computers

Caller makes what looks like a regular function call to the callee, providing parameters if required by the function signature, e.g., `int foo (int x, int y)` requires passing two integers

The callee (foo) receives the parameters, performs some computation, and returns what looks like a regular result if required by the function signature, e.g., `int foo (int x, int y)` requires returning an int

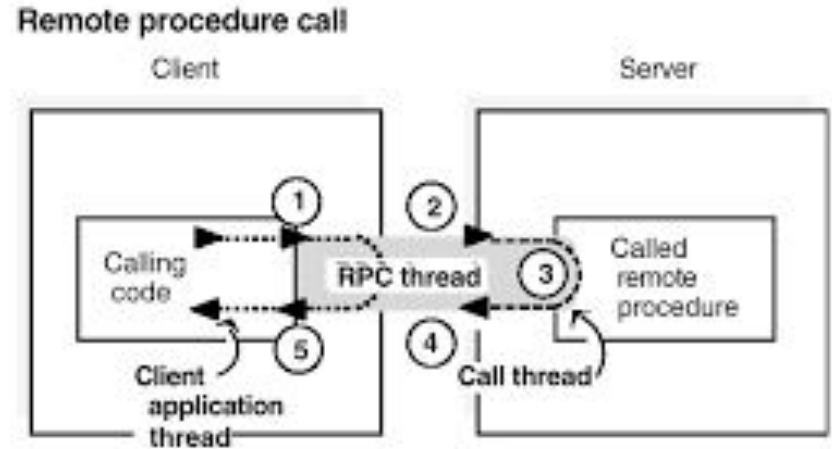
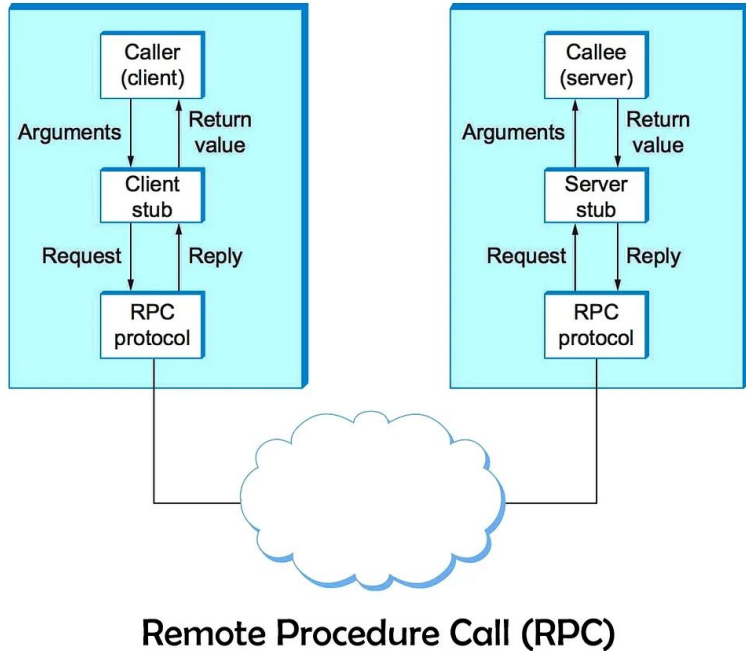


Figure 6-1 Execution Phases of an RPC Thread

The caller and the callee run on different computers, but they do not need to know this - Magic happens under the covers

What Magic?

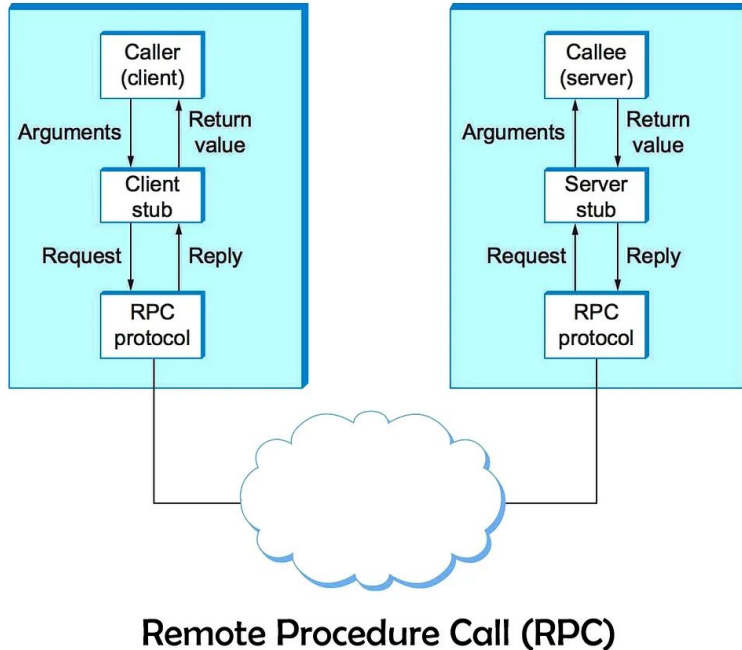


The caller makes a normal programming language call to the 'client stub'

The client stub "serializes" the parameters into some format that is easy to put in network packets

The client stub sends the network packets to the callee via some networking infrastructure

What Magic? 2



Usually, the infrastructure **eventually** delivers the network packets constructed by the client stub to the 'server stub'

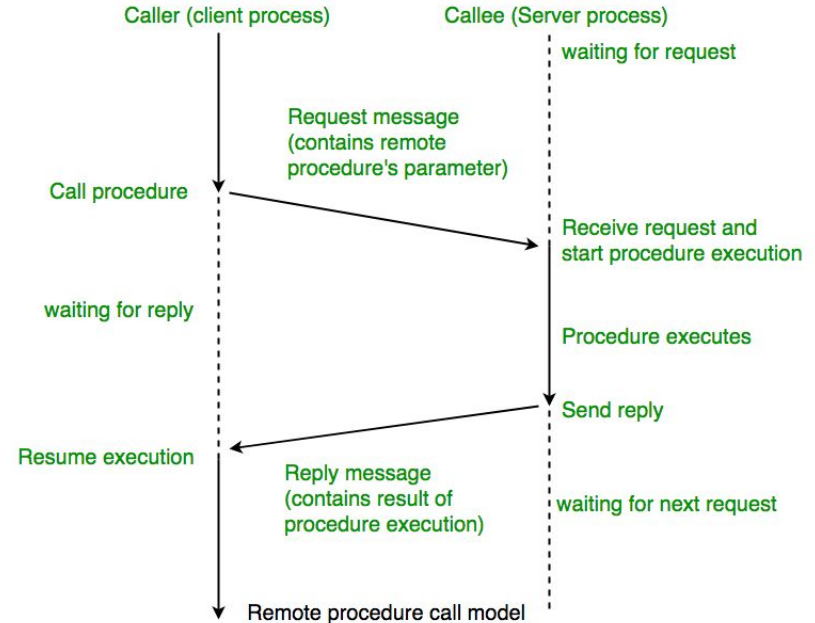
The server stub "deserializes" the parameters into their normal programming language format

The server stub makes a normal programming language call to the callee

What Could Possibly Go Wrong? Client Side

Server running callee is down or unreachable:

- Client stub hangs forever waiting for reply, which means the callee also hangs forever
- Client stub times out and raises exception to caller
- Client stub times out and tries again, some limited number of tries, with the caller still waiting for the return result



What Could Possibly Go Wrong? Server Side



Time out and try again may sound best, but it enables even more failure scenarios:

- Server stub receives request, callee performs computation with side-effects, but the callee, the server stub and/or the server machine crashes before sending response
- Server machine restarts and server stub gets request again, but the callee's computation is not *idempotent* (repeating the call can get a different result)
- Server machine restarts and gets request again, but it's rejected because of anti-replay semantics (security defense)

More failure opportunities if request and/or response are too large for one network packet so spread across several packets (outside scope of this course)

Sometimes there's other servers running copies of the callee, so the infrastructure tries another after timing out on the initial server (and much more can happen here, also outside scope)

Failure Semantics

The caller might never get the return result. Or it might get the return result more than once, possibly different values

Remember I said the caller and the callee run on different computers, *but they do not need to know this*

I lied. They do need to know. This problem arises with RESTful APIs (and the other distributed computing communication patterns to be discussed shortly), not just RPC

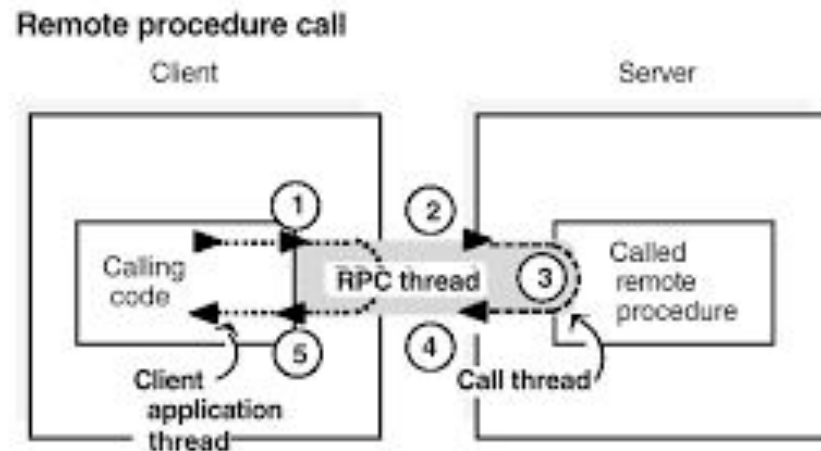


Figure 6-1 Execution Phases of an RPC Thread

Both sides of the communication pattern need code for error handling. Since this is 4156, not 4113, we do not expect your code to deal with it. But your future employer will expect your code (and tests) to deal with it

Distributed Computing Communication Architectures

Client-server or N-tier request-response

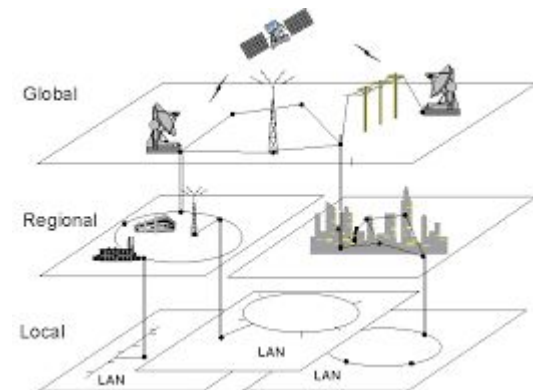
Broker or gateway mediated request-response

Peer to Peer

Service-oriented architecture (SOA)

Publish-subscribe (event-driven)

Others (message queue, manager/worker, blackboard...)

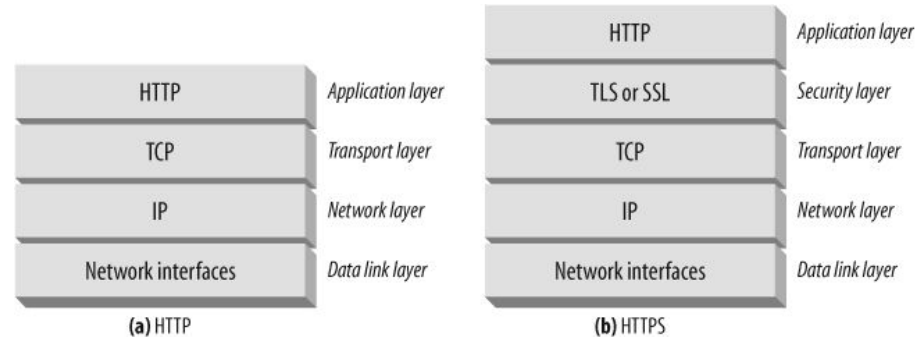
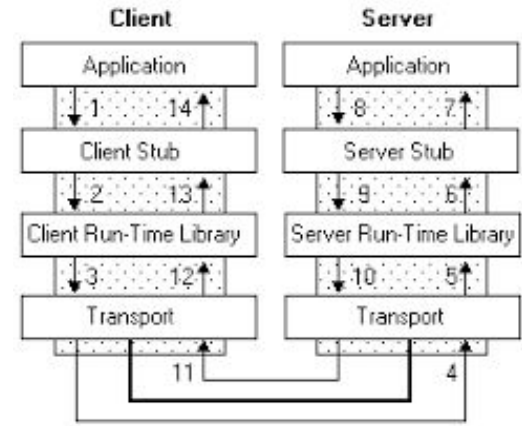


Client-Server Request-Response

Send a request message, wait for response message

Remote procedure call ([RPC](#)):
sending/receiving the message looks like a normal function call, but needs to wait while runtime uses some network transport protocol (and handle failures)

RESTful: a different way to think about RPC that is more data ("[resource](#)") oriented and integrates nicely with HTTP(S) network stack



N-Tier Request-Response

Also send a request message, wait for response message and handle failures, but the message goes through many "tiers" on the way from original requester (client) to final responder (server) and back again

The number and purpose of tiers may change, e.g., adding caches and load balancers, but still conceptually client/server

Tiers generally reside on different hosts, which could be multiple VMs in a datacenter (cloud) that reside on the same or different physical machines

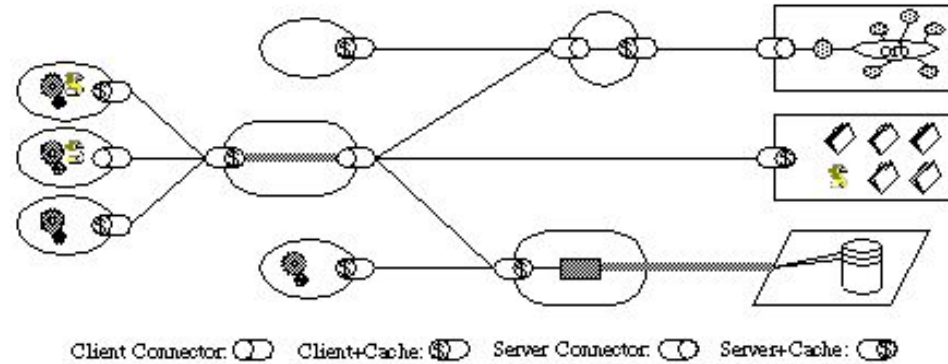
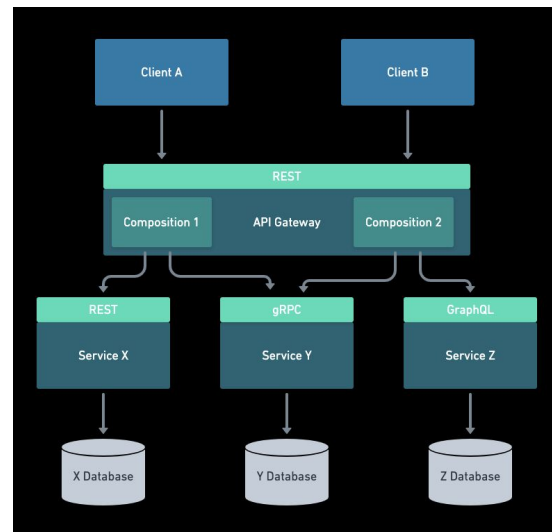
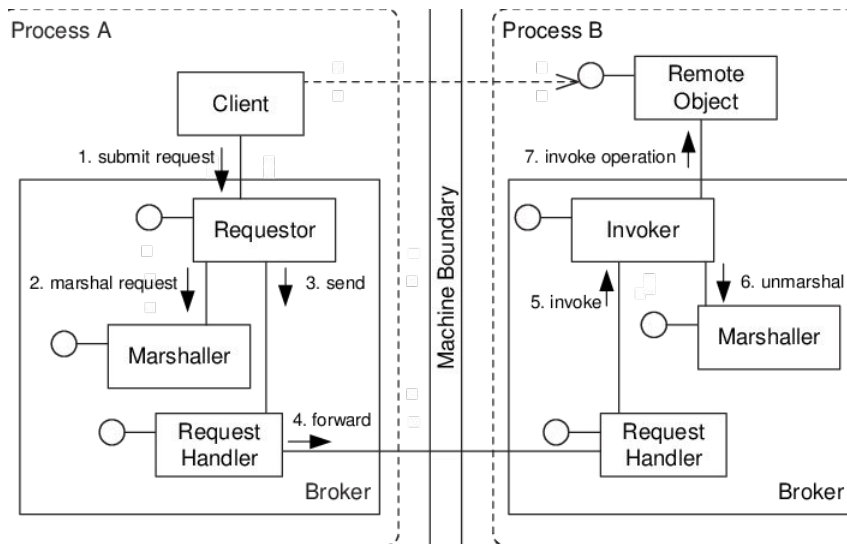
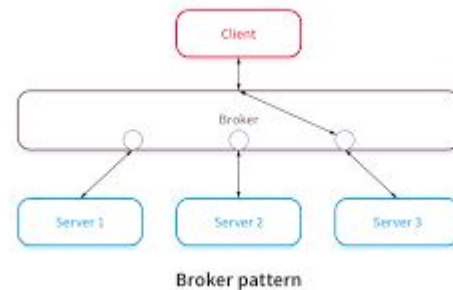


Figure 5-8. REST

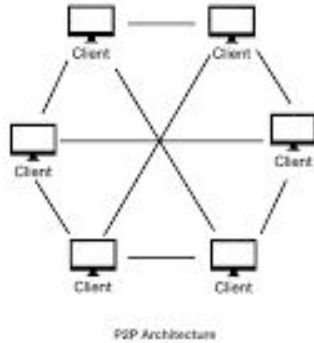
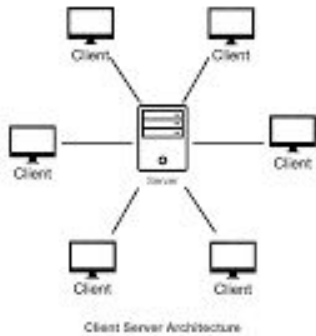


Broker Mediated Request-Response

Essentially same as client-server except the caller machine does not know how to locate the callee machine, and instead sends to a broker or gateway



Peer to Peer (P2P) Computing



In P2P, all the nodes are clients, all the nodes are servers, and all the nodes have APIs

On a single computer, caller A can call function B and function B can call function A

In distributed computing, we usually refer to a caller as a client and the callee as a server. The server sends responses to the client, and there might even be "callbacks", but we don't usually say the server calls the client. But we could... that's peer to peer

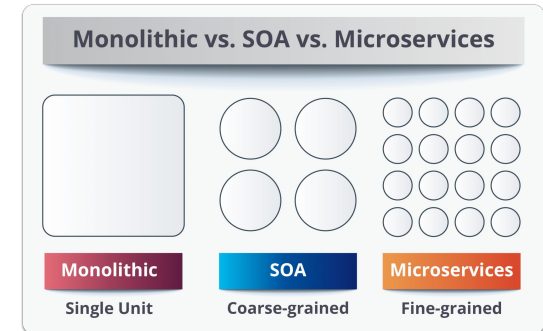
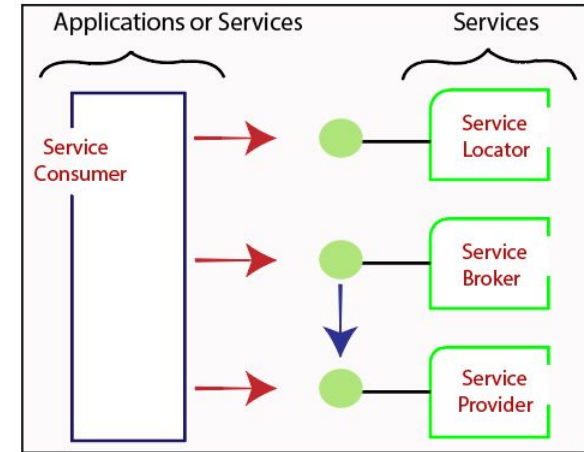
Service-Oriented Architecture (SOA)

SOA is a role-based approach to peer-to-peer: Like P2P, SoA services interact with other services over the network. But peers are more-or-less interchangeable whereas services have predefined purposes

SoA may use both REST and RPC, may use tiers, brokers and gateways

Somewhere there's one or more clients that are apps, not themselves services, for an SOA collection of services → which looks like N-tier to the clients

Not much difference between SOA and microservices, other than “SOA” sounds stodgy and “microservices” sounds cool



Publish-Subscribe



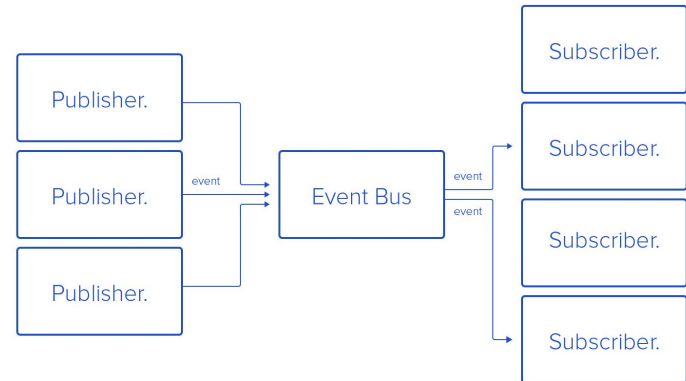
All the other distributed computing architectures were variants of client/server

Pub/sub is different $\sim$ Twitter (X) for software, not humans: the tweets are API calls

aka Event-Driven Architecture

Push - publisher sends notification message or invokes callback trigger for subscribers via a (reverse) broker/gateway

Pull (or polling) - Subscribers keep checking the publisher (or broker/gateway)

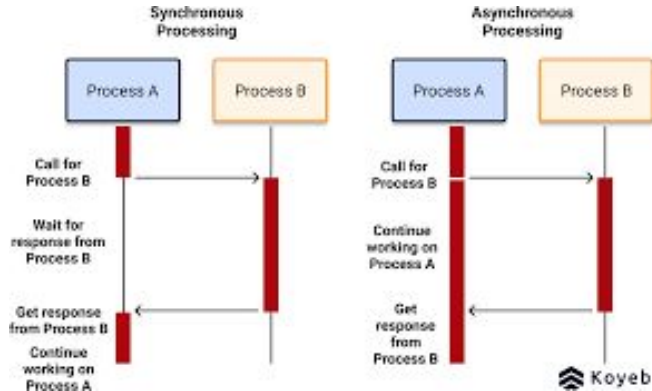


RPC vs. Publish-Subscribe

RPC is (usually) synchronous

Client waits for response from server just like a regular function call

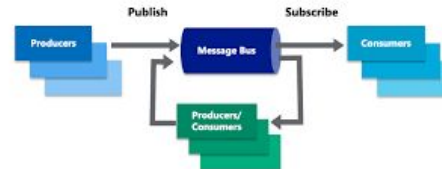
But there needs to be code somewhere to handle networking issues



Publish-subscribe is (usually) asynchronous

Client subscribes to everything from a specific server or everything that matches a specific pattern from a collection of servers

Client may or may not eventually get message(s) from server(s) and may be deluged with many messages

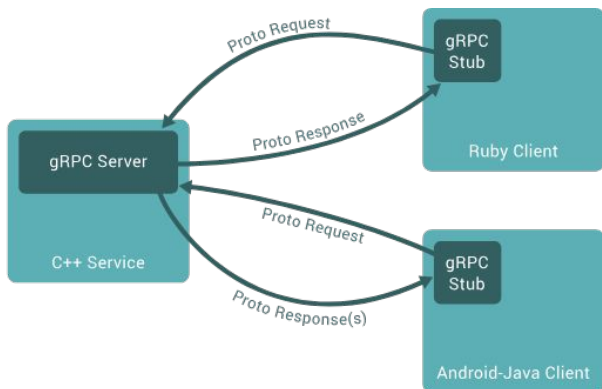


Agenda

1. Distributed Computing communication patterns
2. **RPC vs gRPC**
3. gRPC demo: Justin Tan
4. Next

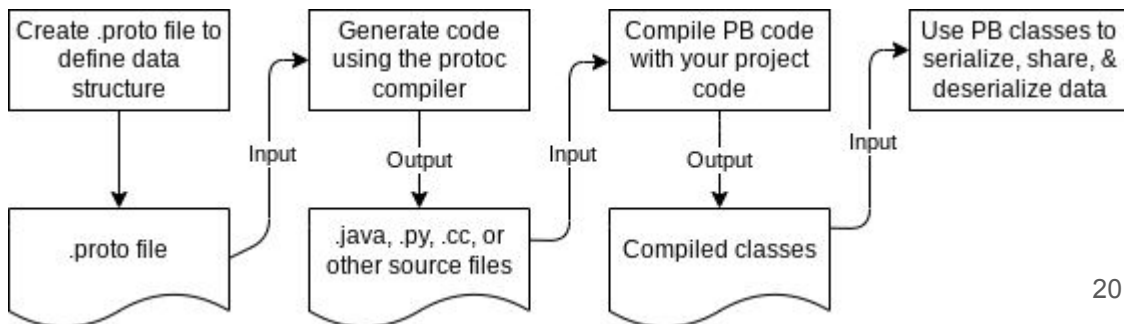


Why gRPC?



RPC frameworks can be language-specific or language-agnostic = support caller and callee written in different programming languages and running on different platforms. There have been, are, and will be many RPC frameworks. We pick gRPC because it's currently popular

gRPC callers and callees communicate using protocol buffers, which serialize structured data into binary form (much faster than json text!). The "proto compiler" generates the client and service code for using protocol buffers



gRPC Service Definition Models

1. Unary - single request and single response

```
rpc SayHello (HelloRequest) returns (HelloResponse);
```

2. Server streaming - single request, stream of responses

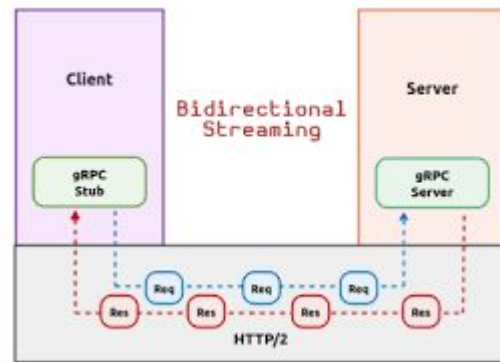
```
rpc LotsOfReplies (HelloRequest) returns (stream HelloResponse);
```

3. Client streaming - stream of requests, single response

```
rpc LotsOfGreetings (stream HelloRequest)  
returns (HelloResponse);
```

4. Bidirectional streaming - streams operate independently and may interleave, with order of messages preserved in each stream

```
rpc BidiHello (stream HelloRequest)  
returns (stream HelloResponse);
```



gRPC Failure Semantics

Deadlines (a fixed point in time) and timeouts (durations of time) may succeed on the server side (“I have sent all my responses!”) but fail on the client side (“The responses arrived after my deadline!”)

Services should specify the longest default deadline they technically support, and clients should wait until the response is no longer useful to them

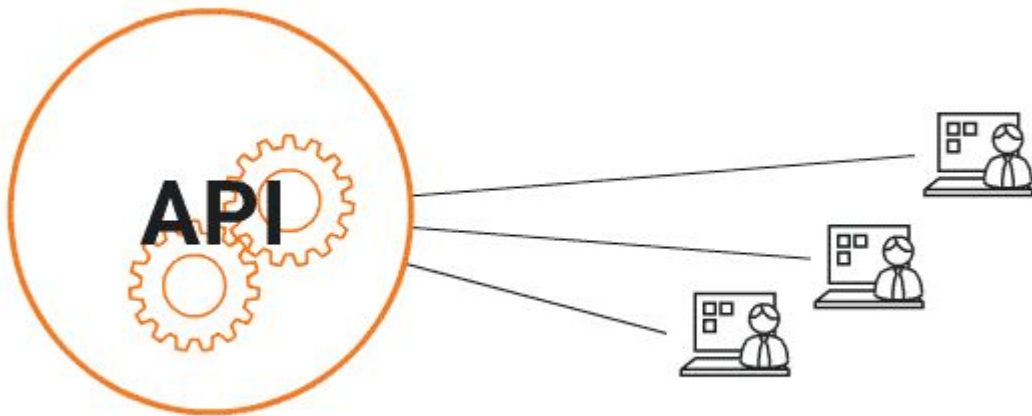
Deadlines can be [adjusted](#) and client requests can be [cancelled](#)

gRPC is [open-source](#) and [well-documented](#), with lots of [learning resources](#)



Agenda

1. Distributed Computing communication patterns
2. RPC vs gRPC
3. gRPC demo: Justin Tan
4. Next



Justin Tan gRPC demo

[demo slides](#)



Agenda

1. Distributed Computing communication patterns
2. RPC vs gRPC
3. gRPC demo: Justin Tan
4. **Next**



Upcoming Deadlines

Meet with your Mentor as soon as you can!

[Team First Iteration](#) due Friday October 18

[Team First Iteration Demo](#) due Friday October 25



Thursday

working in teams

project management tools demo



Ask Me Anything

